🖥️ 02 — Client-Server Model

📖 Introduction

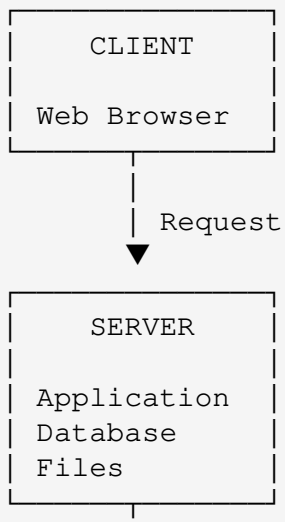
After learning how websites work, the next concept I want to understand is the **Client-Server Model**.

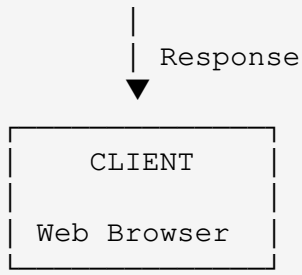
Most websites and web applications work because two main sides communicate with each other:

- **Client**
- **Server**

The client requests something, and the server processes the request and sends a response.

A simplified model looks like this:





1️⃣ What Is a Client?

A **client** is a device or application that requests a service or resource from another computer.

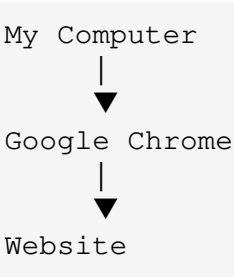
When browsing the web, the client is usually the **web browser**.

Examples include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
- Mobile web browsers

The device running the browser can also be considered the client.

For example:



The browser sends requests to servers and receives responses.

2️⃣ What Is a Server?

A **server** is a computer or system that provides resources or services to clients.

A web server can provide things such as:

- HTML files
- CSS files
- JavaScript files
- Images
- Videos
- APIs
- Application data

For example:

```
Client
   |
   | Request
   ▼
Web Server
   |
   | Response
   ▼
Client
```

A server is not necessarily a special type of computer.

A server is essentially a computer/system performing a role of providing services or resources to other systems.

3️⃣ Client vs Server

The main difference is their role.

Client	Server
Requests resources	Provides resources
Usually used by the user	Usually runs services
Sends requests	Receives requests
Receives responses	Sends responses
Example: Browser	Example: Web server

A simple way to remember this:

> **Client asks. Server responds.**

4️⃣ How They Communicate

The client and server communicate over a network.

For websites, this communication commonly happens using **HTTP or HTTPS**.

The process can be simplified as:

```
Client
  |
  | HTTP Request
  ▼
Server
  |
  | HTTP Response
```

```
   ▼
Client
```

5️⃣ Example: Opening a Website

Imagine I enter:

```
https://example.com
```

into my browser.

The simplified process is:

```
1. I enter the URL
        ↓
2. Browser determines where to connect
        ↓
3. Browser sends a request
        ↓
4. Server receives the request
        ↓
5. Server processes it
        ↓
6. Server sends a response
        ↓
7. Browser receives the response
        ↓
8. Browser displays the webpage
```

6️⃣ What Is a Request?

A **request** is a message sent by the client asking the server to perform an action or provide a resource.

For example, the browser may request:

```
GET /
```

This can be interpreted as:

> "Give me the resource located at `/`."

Another example:

```
GET /about
```

The client is requesting the `/about` resource.

7️⃣ What Is a Response?

A **response** is the message sent back by the server.

For example:

```
HTTP/1.1 200 OK
```

The server can then return content such as HTML.

Simplified:

```
CLIENT
   |
   | GET /about
   ▼
SERVER
```

```
    |
    | 200 OK
    | HTML
    ▼
CLIENT
```

8️⃣ HTTP Methods

HTTP provides different methods that clients can use when communicating with servers.

Some common methods are:

Method	Purpose
GET	Retrieve data
POST	Send/create data
PUT	Replace/update data
PATCH	Partially update data
DELETE	Delete data

For example:

```
GET
↓
"Give me this information."

POST
↓
"Here is some information. Create something with it."

PUT
↓
```

```
"Replace this existing resource."

PATCH
↓
"Update part of this resource."

DELETE
↓
"Remove this resource."
```

9️⃣ Client-Server Example: Login

A login system demonstrates the client-server model very well.

Imagine I enter:

```
Username: mukhtaar
Password: ********
```

and click **Login**.

The process could look like:

```
              CLIENT
                 │
                 │ Login credentials
                 ▼
              SERVER
                 │
                 │ Check credentials
                 ▼
            DATABASE
                 │
                 │ User found?
```

```
                ▼
            SERVER
                │
                │ Authentication result
                ▼
            CLIENT
```

The browser does not normally need direct access to the database.

Instead:

```
Client
  ↓
Server
  ↓
Database
  ↓
Server
  ↓
Client
```

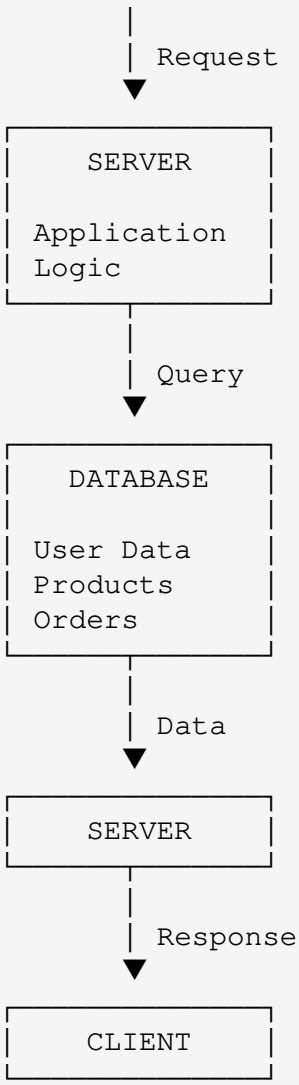
The server acts as an intermediary.

🔟 Client-Server With a Database

Many modern applications have more than just a client and a server.

A simplified application architecture might look like:

```
┌──────────────┐
│    CLIENT    │
│              │
│ Browser/App  │
└──────┬───────┘
```



This architecture is common in web applications.

---

10 Why Do We Need Servers?

Servers allow applications to provide centralized services and resources.

For example, an application might use a server to:

- Store user accounts
- Process payments
- Authenticate users
- Store files
- Process business logic
- Access databases
- Provide APIs
- Serve webpages

Without servers, many modern online applications would not function in the same way.

1□2□Are Servers Always Far Away?

No.

A server could be:

- In another country
- In the same city
- In a company's office
- In a data center
- On a local network
- Running on my own computer

For example, during web development I can run a local server on my computer:

```
My Computer
     |
```

```
      ├── Browser
      │
      └── Local Web Server
```

I can then access something like:

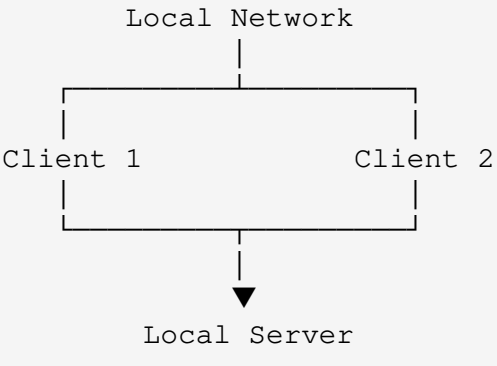
```
http://localhost
```

The request never needs to travel to a public internet server.

1️⃣3️⃣ Client-Server on a Local Network

Client-server communication also happens inside local networks.

For example:



This is relevant to IT infrastructure and networking because client-server architecture is not limited to the public internet.

The Most Important Concept

The key idea I learned from the client-server model is:

```
CLIENT
  |
  | Request
  ▼
SERVER
  |
  | Processing
  ▼
SERVER
  |
  | Response
  ▼
CLIENT
```

The client **requests**.

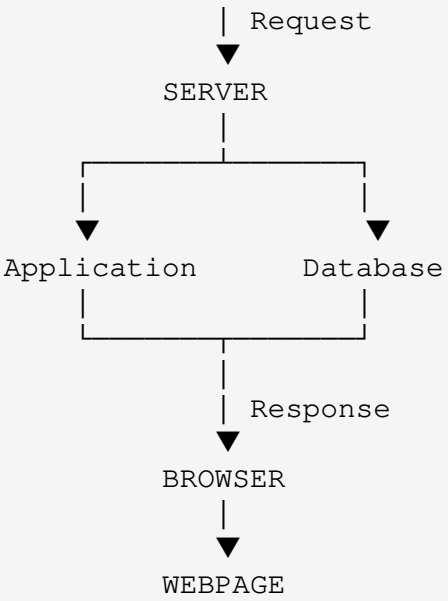
The server **processes**.

The server **responds**.

🔄 Complete Example

When I visit an online store:

```
USER
  |
  ▼
BROWSER
  |
```



The browser is the client.

The server handles the application logic.

The database stores information.

What I Learned

From this topic, I learned that websites and web applications often operate using a **client-server architecture**.

The client is responsible for requesting resources and displaying information to the user.

The server receives requests, performs processing, accesses resources such as databases when necessary, and sends responses back to the client.

I also learned that:

- A browser can act as a client.
- A server provides services or resources.
- Clients and servers communicate through networks.
- HTTP/HTTPS are commonly used for web communication.
- Servers can communicate with databases.
- Client-server architecture can exist on the internet or a local network.

 Key Terms

Term	Meaning
Client	System that requests a service or resource
Server	System that provides a service or resource
Request	Message sent from client to server
Response	Message sent from server to client
HTTP	Protocol used for web communication
HTTPS	Secure version of HTTP using TLS
Database	System used to store and manage data
API	Interface that allows software systems to communicate

? Questions I Want to Explore Next

This topic raises some new questions:

- How does the browser know which server to contact?
- How exactly does DNS work?
- What happens inside a web browser?
- What is a web server?
- What is the difference between a web server and an application server?
- How does HTTP work?
- How does HTTPS protect communication?
- How does a server communicate with a database?

These will lead into the next Web Fundamentals topics.

📈 Progress

Stage: Web Fundamentals

Topic: Client-Server Model

Status: 🟢 Completed

Previous: How Websites Work

Next: Web Browsers

> **A website is not just a page on a screen — it is the result of systems communicating with each other.**